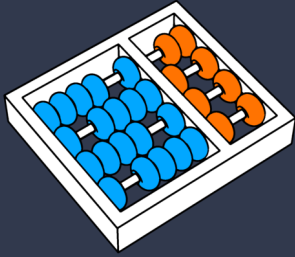




UNICAMP



Introdução ao Langchain

Prof. Dr. André Gomes Regino

regino@unicamp.br

Agenda



01 Motivação: por que precisamos de frameworks?

02 O que é LangChain

03 Arquitetura: Chains, Tools, Memory, Agents

04 Exemplos práticos de código

05 Alternativas ao LangChain

06 Quando usar (e quando não usar)

Limitações e Objetivos

Limitação

Técnicas como prompt engineering, few-shot learning e self-consistency aumentam a qualidade das respostas, mas são aplicadas de forma manual e fragmentada.

- Cada chamada ao LLM é um **ponto isolado no código**
- **Não há padronização** de entradas/saídas
- Reprodutibilidade comprometida
- **Difícil auditar** ou versionar o comportamento do sistema

Limitação

Aplicações reais NÃO são

- 1 prompt → 1 resposta

Elas envolvem

- múltiplos passos
- dados externos
- memória
- decisões

“LLM sozinho não resolve sistema”

Limitação

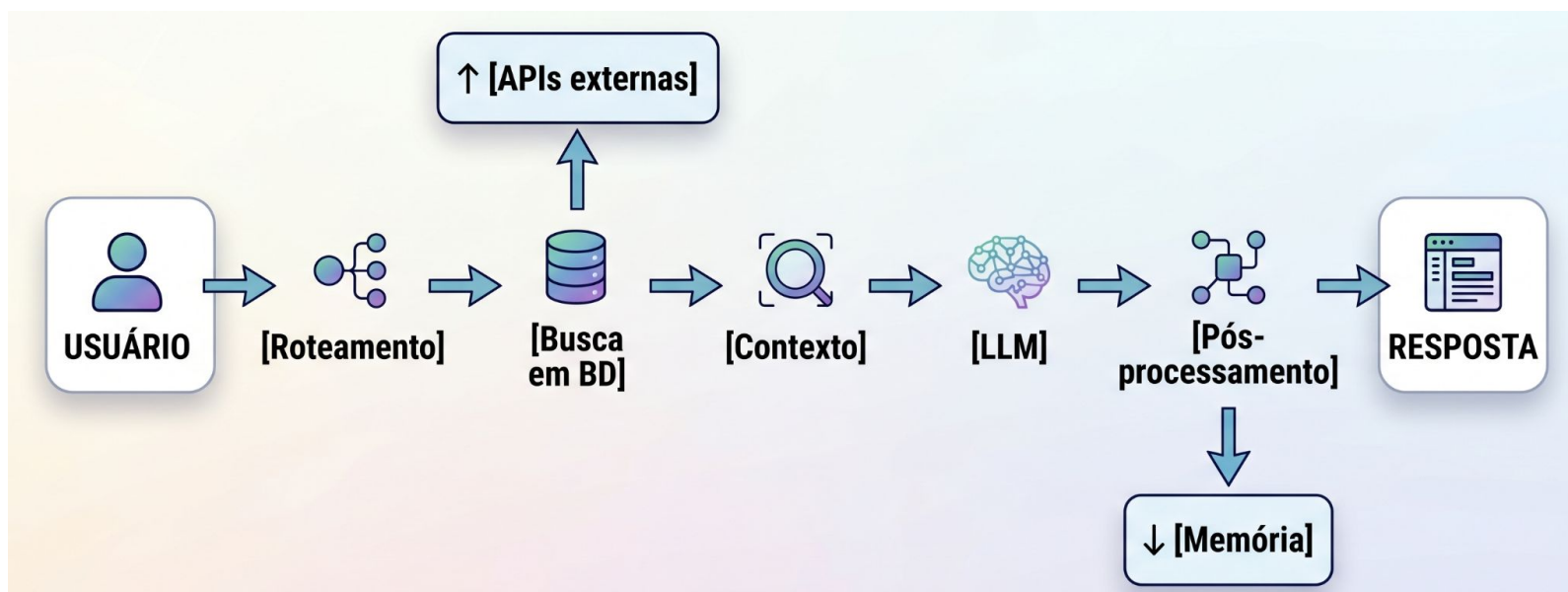
A DIFERENÇA: LLM VIA CHAT vs. LLM CONECTADA A SISTEMAS



Limitação

Um LLM sozinho responde com **o que sabe**

Um sistema responde com **o que sabe** + **o que busca** + **o que lembra**



Exemplo

Chatbot de e-commerce

Pergunta: "Qual o prazo de entrega do produto X para o CEP Y?"

1. Identificar intenção

Analisa o que o usuário deseja realizar no chat.

Classificador / LLM

2. Buscar produto

Localiza as especificações técnicas no catálogo.

Tool (SQL/API)

3. Consultar frete

Integra com serviços externos (ex: Correios).

Tool (HTTP)

4. Recuperar histórico

Busca interações passadas para manter o contexto.

Memory

5. Gerar resposta

Consolida todos os dados em um texto natural.

Chain (prompt + LLM)

6. Logar interação

Registra a saída para auditoria e melhoria contínua.

Output parser

Limitação

Sem framework

- Código desorganizado
- Difícil manutenção
- Sem reuso
- Difícil escalar

Surge a necessidade de orquestração

Objetivos

- **Explicar** o papel do LangChain na construção de sistemas com LLMs
- **Identificar** os 4 componentes centrais da arquitetura
- **Escrever** uma chain básica
- **Comparar** LangChain com alternativas
- **Decidir** quando usar ou não o framework em um projeto real

O que é Langchain

construindo sistemas, não só prompts

Definição

- Framework open-source para **desenvolvimento de aplicações baseadas em LLMs**, criado em outubro de 2022
- Disponível em Python e JavaScript/TypeScript
- GitHub: github.com/langchain-ai/langchain (+100k ★)
- A proposta central: construir sistemas, não prompts

Benefícios



Orquestração

Coordena chamadas ao LLM, ferramentas e memória em um fluxo único.



Modularização

Cada etapa do pipeline é um componente trocável independentemente.



Reuso

Uma chain pode ser reutilizada em múltiplos projetos.



Integração

Conecta nativamente a BDs, PDFs, APIs e vetoriais como Pinecone, Chroma e FAISS.

Integrações e Casos de Uso

Integrações

OpenAI, Anthropic, Google Gemini, HuggingFace, Pinecone, Weaviate, PostgreSQL, MongoDB, Slack, Gmail, YouTube, Notion, e +700 integrações

Casos de Uso



Chatbots



RAG



Automação



Agentes

Arquitetura

Componentes



Chains

Sequência de passos e fluxos de execução.



Tools

Funções externas para busca e processamento.



Memory

Persistência de estado e contexto de chat.



Agents

Tomada de decisão e orquestração autônoma.

Chains

- pipelines compostos com *LCEL* (*LangChain Expression Language*)

```
1 from langchain_core.output_parsers import StrOutputParser
2
3 # Chain 1: gera pergunta de acompanhamento
4 chain_followup = prompt_followup | llm | StrOutputParser()
5
6 # Chain 2: responde com base na pergunta gerada
7 chain_resposta = prompt_resposta | llm | StrOutputParser()
8
9 # Chain sequencial: saída de uma vira entrada da próxima
10 pipeline = chain_followup | chain_resposta
11
12 resultado = pipeline.invoke({"topico": "machine learning"})
```

Tools

- Funções que o LLM pode decidir chamar
- O LLM não executa as tools, só qual chamar e com quais argumentos
 - A execução é do framework

```
1 from langchain_core.tools import tool
2
3 @tool
4 def buscar_preco(produto: str) -> str:
5     """Busca o preço atual de um produto no banco de dados."""
6     return db.query(f"SELECT preco FROM produtos WHERE nome='{produto}'")
7
8 @tool
9 def calcular_frete(cep: str, peso_kg: float) -> str:
10     """Calcula o frete para um CEP dado o peso do produto."""
11     return api_correios.calcular(cep, peso_kg)
12
13 tools = [buscar_preco, calcular_frete]
14 llm_com_tools = llm.bind_tools(tools)
```

Memory

- Guarda contexto

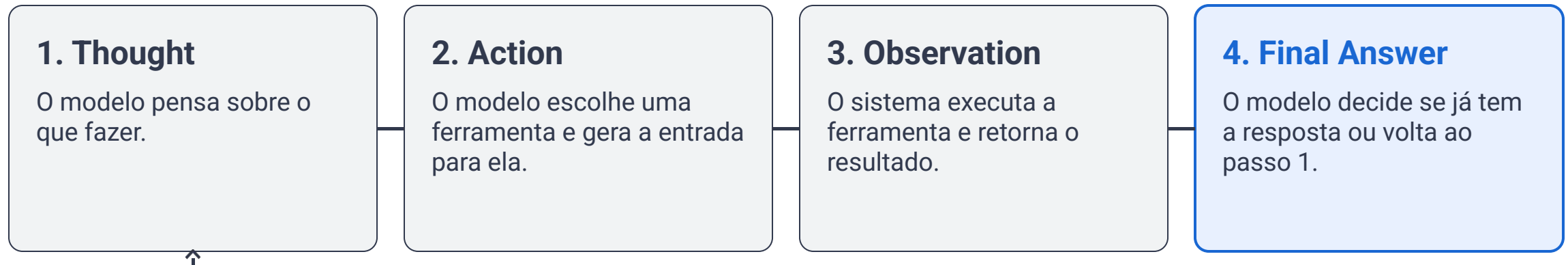
Tipo	Funcionamento	Custo de tokens
ConversationBufferMemory	Mantém todo o histórico	Alto (cresce infinito)
ConversationBufferWindowMemory	Últimas N mensagens	Controlado
ConversationSummaryMemory	Resume o histórico com LLM	Médio

```
1 from langchain.memory import ConversationBufferMemory
2
3 memory = ConversationBufferMemory(return_messages=True)
4 memory.save_context({"input": "Oi, sou o André"}, {"output": "Olá André!"})
```

Agents

Agentes usam o LLM como "**raciocinador**".
Diferente de uma Chain (fluxo fixo), o Agente decide o próximo passo **dinamicamente**.

O Loop do Agente (ReAct)



Exemplo 1 – Chain simples

```
1 from langchain_openai import ChatOpenAI
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_core.output_parsers import JsonOutputParser
4 from pydantic import BaseModel
5
6 class RespostaEstruturada(BaseModel):
7     resposta: str
8     confianca: float
9     fontes: list[str]
10
11 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
12 parser = JsonOutputParser(pydantic_object=RespostaEstruturada)
13
14 prompt = ChatPromptTemplate.from_messages([
15     ("system", "Responda em JSON válido seguindo o schema: {format_instructions}"),
16     ("human", "{pergunta}")
17 ]).partial(format_instructions=parser.get_format_instructions())
18
19 chain = prompt | llm | parser
20
21 resultado = chain.invoke({"pergunta": "O que é backpropagation?"})
22 print(resultado) # dict com resposta, confiança e fontes
```

Exemplo 2 – RAG básico

```
1 from langchain_community.document_loaders import PyPDFLoader
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3 from langchain_openai import OpenAIEmbeddings
4 from langchain_community.vectorstores import FAISS
5 from langchain.chains import RetrievalQA
6
7 # 1. Carregar documento
8 loader = PyPDFLoader("artigo.pdf")
9 docs = loader.load()
10
11 # 2. Dividir em chunks
12 splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
13 chunks = splitter.split_documents(docs)
14
15 # 3. Criar índice vetorial
16 embeddings = OpenAIEmbeddings()
17 vectorstore = FAISS.from_documents(chunks, embeddings)
18
19 # 4. Chain RAG
20 chain = RetrievalQA.from_chain_type(
21     llm=ChatOpenAI(model="gpt-4o"),
22     retriever=vectorstore.as_retriever(search_kwargs={"k": 4})
23 )
24
25 print(chain.invoke("Quais são as conclusões do artigo?"))
```

Alternativas

Por que existem alternativas

Desafios do Crescimento Rápido

- **Instabilidade da API:** Mudanças drásticas entre versões (v0.0.x → v1.0)
- **Complexidade Oculta:** Abstrações excessivas dificultam o entendimento
- **Dificuldade de Debug:** Processo complexo
- **Crítica da Comunidade:** "LangChain is unnecessarily complex for most use cases" (HackerNews, 2024).

Resposta da Comunidade: Alternativas mais enxutas e especializadas.

Maturidade: A existência de opções indica um ecossistema saudável e em evolução.

LlamaIndex

	LangChain	LlamaIndex
Foco	Orquestração geral	Dados + RAG
Curva de aprendizado	Alta	Média
RAG avançado	Requer mais código	Nativo
Agentes	Sim	Sim (menos maduro)

Semantic Kernel

	LangChain	Semantic Kernel
Mantenedor	LangChain AI	Microsoft
Linguagens	Python, JS	Python, C#, Java
Melhor para	Projetos Python	Stacks Microsoft

Quando usar: ambientes corporativos já integrados ao ecossistema Microsoft/Azure.

Ecosystema

Ecossistema e Documentação

- Plugins
- Integrações
- Comunidade ativa

langchain-core	→ interfaces base (Runnable, BaseTool, etc.)
langchain	→ chains, agents, memory de alto nível
langchain-community	→ integrações da comunidade (700+ conectores)
langchain-openai	→ integração oficial OpenAI
langgraph	→ agentes com estado como grafos
langsmith	→ observabilidade, avaliação e debug

Documentação

- Docs oficiais (python.langchain.com/docs)
- Cursos (academy.langchain.com)
- GitHub (github.com/langchain-ai/langchain)
- Artigos
 - *Lewis et al. (2020). RAG for Knowledge-Intensive NLP. NeurIPS*
 - *Yao et al. (2022). ReAct. ICLR 2023*

Estratégias

Nem sempre usar LangChain

Situação	Alternativa mais simples
1 único prompt, 1 resposta	Chamar a API diretamente
Script de processamento em lote	openai lib + loop Python
Prototipagem rápida (< 1 dia)	Notebooks + API direta
Equipe sem familiaridade	Curva de aprendizado supera o ganho



Regra prática: se você precisaria de menos de 50 linhas sem o framework, pule o framework.

Nem sempre usar LangChain

```
1 # Isso NÃO precisa de LangChain:
2 from openai import OpenAI
3 client = OpenAI()
4 resp = client.chat.completions.create(
5     model="gpt-4o",
6     messages=[{"role": "user", "content": "Resuma este texto: ..."}]
7 )
8 print(resp.choices[0].message.content)
```



Regra prática: se você precisaria de menos de 50 linhas sem o framework, pule o framework.

Quando Usar

Use LangChain quando:



Múltiplos Passos

Sistema com dependências complexas entre etapas.



Ferramentas Externas

Integração necessária com APIs, Bancos de Dados e buscas.



Fluxo Dinâmico

Comportamento onde o agent decide o próximo passo.



RAG Avançado

Necessidade de RAG utilizando múltiplas fontes de dados.



Escalabilidade

A equipe pretende manter e escalar o sistema a longo prazo.

Instabilidades do Langchain

Lembrete Final Importante

Esta aula foca em **entender o básico de Langchain**.

Se um código quebrar no seu ambiente, o problema quase sempre é de **versão**, não de lógica.

Entendendo o racional, você será capaz de **adaptar o código** a qualquer versão futura.

Take-home Message



Orquestração é Chave

LLMs sozinhos não constroem sistemas; a orquestração une as peças.



Blocos de Construção

Chains, Tools, Memory e Agents são os fundamentos do framework.



LCEL & Composição

Domine o operador `|` para compor fluxos robustos.



Padrão RAG

LangChain + VectorStore + LLM é o padrão dominante em produção.



Alternativas

Considere LlamaIndex para dados ou Semantic Kernel para o ecossistema MS.



Use com Critério

Frameworks têm custo de abstração; use apenas quando necessário.

"The goal is not to use a framework. The goal is to build a system that works."